

Comunicación en tiempo real entre el navegador y el servidor

Tecnología a Desarrollar: **WebSocket**

Integrantes:

Franco Pierabella Franco Pierabella 53755

Fabrizio Cristofoli fabriciodamiancristofoli@gmail.com

Joaquin Guarc

Iara Melano

Comisión 301 DSW 2026

Introducción

Las aplicaciones web han pasado de ofrecer contenido estático a brindar experiencias interactivas que requieren comunicación inmediata entre clientes y servidores. Plataformas de mensajería instantánea, videojuegos en línea, aplicaciones colaborativas, sistemas de monitoreo y paneles en tiempo real son algunos ejemplos donde la velocidad de intercambio de información resulta un factor determinante para la experiencia del usuario.

La comunicación entre un navegador y un servidor se basa en el protocolo HTTP, que funciona como un esquema de solicitud y respuesta (Request–Response). Bajo este modelo, el cliente debe iniciar cada intercambio de información, mientras que el servidor únicamente responde a dichas solicitudes. Aunque este mecanismo resulta adecuado para una gran variedad de aplicaciones, presenta limitaciones cuando se requiere transmitir información de forma continua y con baja latencia, ya que obliga al cliente a consultar periódicamente al servidor para detectar cambios.

Como respuesta a estas necesidades surgieron diversas tecnologías orientadas a la comunicación en tiempo real, entre las que se destacan WebSocket, Server-Sent Events (SSE), WebRTC y MQTT. Cada una de estas tecnologías fue diseñada para resolver diferentes problemáticas y presenta características particulares que la hacen más adecuada para determinados escenarios de uso.

En el marco de este informe, se investigará la tecnología WebSocket con el objetivo de comprender su funcionamiento, analizar sus principales características técnicas, evaluar sus ventajas y limitaciones, e implementar una demostración práctica que permita validar su utilización en un escenario de comunicación en tiempo real. Asimismo, se establecerán criterios de comparación con las tecnologías SSE, WebRTC y MQTT, con el propósito de determinar en qué situaciones resulta conveniente emplear WebSocket y cuáles son sus diferencias respecto de las alternativas disponibles.

Modelo tradicional de comunicación: HTTP

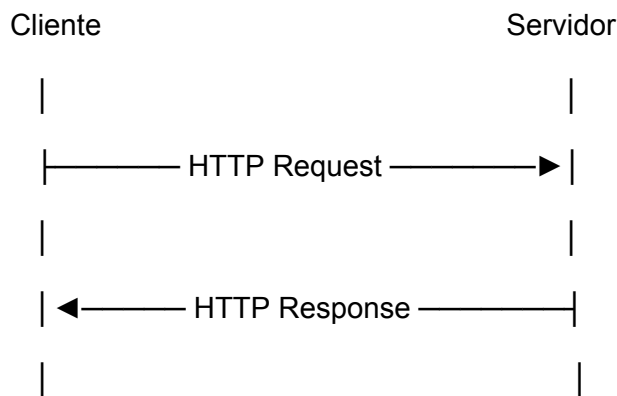
Antes de comprender el funcionamiento de WebSocket, resulta necesario analizar cómo se comunican tradicionalmente las aplicaciones web mediante el protocolo HTTP (HyperText Transfer Protocol), ya que esta arquitectura constituye el punto de partida sobre el cual surgieron las tecnologías de comunicación en tiempo real.

Modelo Cliente – Servidor

HTTP es un protocolo de comunicación basado en un modelo de solicitud y respuesta (*Request–Response*). En este esquema, el cliente (generalmente un navegador web) inicia una solicitud hacia el servidor, el cual procesa la petición y devuelve una respuesta con la información requerida. Una vez enviada la respuesta, la comunicación finaliza y la conexión puede cerrarse.

Este modelo resulta adecuado para la mayoría de las aplicaciones web tradicionales, donde el usuario realiza una acción específica —como acceder a una página, enviar un formulario o consultar información— y espera una respuesta del servidor antes de continuar interactuando con la aplicación.

El flujo básico de comunicación puede representarse de la siguiente manera:



(La comunicación finaliza)

En este modelo, toda comunicación depende de que el cliente inicie una nueva solicitud. El servidor no puede enviar información de forma espontánea, incluso si dispone de nuevos datos que serían relevantes para el usuario.

Limitaciones del modelo HTTP

La principal limitación radica en que cada intercambio de información implica una nueva solicitud iniciada por el cliente. Es decir, si una aplicación necesita conocer cambios que ocurren constantemente en el servidor, deberá enviar solicitudes de manera periódica para consultar si existe información nueva.

Este comportamiento genera distintos inconvenientes:

- Incremento en la cantidad de solicitudes HTTP realizadas entre cliente y servidor.
- Mayor consumo de ancho de banda debido al envío repetitivo de encabezados HTTP.
- Aumento de la carga de procesamiento tanto en el cliente como en el servidor.

- Mayor latencia entre el momento en que ocurre un evento y el instante en que el usuario recibe la actualización.
- Imposibilidad de que el servidor notifique cambios inmediatamente sin una solicitud previa del cliente.

Hacia una comunicación verdaderamente en tiempo real

El crecimiento de aplicaciones que requerían intercambios constantes de información evidenció que las soluciones basadas exclusivamente en HTTP no resultaban suficientes para determinados escenarios. Era necesario un mecanismo que permitiera mantener una conexión activa entre cliente y servidor, eliminando la necesidad de crear una nueva comunicación para cada mensaje intercambiado.

En este contexto surgió el protocolo WebSocket, diseñado para establecer una conexión persistente y bidireccional que posibilita el intercambio de información en tiempo real con una menor latencia y una utilización más eficiente de los recursos de red. Esta evolución representa un cambio significativo respecto al modelo tradicional de comunicación y constituye la base de numerosas aplicaciones modernas.

WebSocket: una evolución del modelo de comunicación web

Las limitaciones del modelo tradicional basado en HTTP y las restricciones de técnicas evidenciaron la necesidad de un mecanismo que permitiera mantener una comunicación continua entre clientes y servidores sin generar nuevas solicitudes para cada intercambio de información. Como respuesta a esta problemática surgió WebSocket, un protocolo diseñado específicamente para establecer conexiones persistentes y bidireccionales, sentando las bases de gran parte de las aplicaciones web modernas que requieren comunicación en tiempo real.

¿Qué es WebSocket?

WebSocket es un protocolo de comunicación que permite establecer una conexión persistente y bidireccional entre un cliente y un servidor a través de una única conexión TCP. A diferencia del modelo tradicional de HTTP, donde cada intercambio de información requiere una nueva solicitud iniciada por el cliente, WebSocket mantiene la conexión abierta durante toda la comunicación. Esto permite que tanto el cliente como el servidor puedan intercambiar mensajes en cualquier momento, sin necesidad de establecer nuevas conexiones para cada transmisión.

Gracias a esta característica, WebSocket reduce significativamente la latencia y la sobrecarga asociada al protocolo HTTP, convirtiéndose en una solución especialmente adecuada para aplicaciones donde la información debe actualizarse de manera inmediata.

Es importante destacar que WebSocket no reemplaza completamente a HTTP. De hecho, una conexión WebSocket comienza mediante una solicitud HTTP especial denominada **Handshake**, utilizada para negociar el cambio de protocolo entre el cliente y el servidor. Una vez establecida la conexión, la comunicación deja de utilizar HTTP y pasa a realizarse mediante el protocolo WebSocket.

Características principales

WebSocket presenta un conjunto de características que lo diferencian del modelo tradicional basado en HTTP y que justifican su amplia adopción en aplicaciones que requieren intercambio continuo de información.

Conexión persistente: Una vez establecido el Handshake inicial, la conexión permanece abierta hasta que alguno de los extremos decide finalizarla. Esto evita la creación de nuevas conexiones para cada mensaje y reduce considerablemente el tiempo necesario para intercambiar información.

Comunicación Full-Duplex: WebSocket permite que cliente y servidor envíen mensajes de manera independiente y simultánea. Ambos participantes pueden transmitir información cuando lo consideren necesario, sin depender de solicitudes previas ni esperar que la otra parte finalice una operación. Esta característica resulta fundamental para aplicaciones donde múltiples eventos pueden producirse de forma concurrente, como chats, videojuegos o herramientas colaborativas.

Baja latencia: Al mantener una conexión permanente y eliminar la necesidad de establecer múltiples solicitudes HTTP, el intercambio de mensajes se realiza prácticamente de forma inmediata, reduciendo la latencia percibida por el usuario.

Comunicación eficiente: Los mensajes intercambiados mediante WebSocket poseen una estructura considerablemente más liviana que una solicitud HTTP completa. Como consecuencia, disminuye la cantidad de información transmitida en cada intercambio y se optimiza el uso del ancho de banda, especialmente cuando la comunicación es frecuente.

Comunicación basada en eventos: Otra característica destacable es que la comunicación deja de depender del modelo tradicional de solicitud y respuesta. Tanto el cliente como el servidor reaccionan a eventos, enviando información únicamente cuando ocurre una acción relevante, lo que favorece aplicaciones dinámicas e interactivas.

Arquitectura general

Desde una perspectiva general, WebSocket mantiene el modelo Cliente–Servidor característico de las aplicaciones web. Sin embargo, a diferencia de HTTP, la comunicación se establece mediante una única conexión persistente que permanece activa durante toda la sesión.

Una vez creada la conexión, cliente y servidor pueden intercambiar mensajes libremente, sin necesidad de renegociar la comunicación para cada intercambio de información.

Esta arquitectura permite que el servidor envíe datos inmediatamente cuando ocurre un evento, eliminando la necesidad de que el cliente consulte constantemente si existen novedades.

¿Por qué representa una evolución respecto a HTTP?

Mientras HTTP establece una nueva comunicación para cada solicitud realizada por el cliente, WebSocket mantiene una conexión persistente que permanece activa durante toda la sesión. Esto reduce la sobrecarga de la comunicación, disminuye la latencia y permite que el servidor transmita información de manera espontánea cuando se producen cambios relevantes. Ambos protocolos fueron diseñados para resolver necesidades diferentes. HTTP continúa siendo la opción más adecuada para la mayoría de las operaciones tradicionales de una aplicación web, como la obtención de páginas, el envío de formularios o el acceso a servicios REST.

WebSocket complementa este modelo al ofrecer un canal de comunicación permanente para aquellas funcionalidades que requieren intercambio continuo de información en tiempo real. En consecuencia, muchas aplicaciones modernas utilizan ambos protocolos de manera conjunta, aprovechando las fortalezas de cada uno según las necesidades del sistema.

WebSocket en una aplicación real

Hasta este punto se ha explicado por qué surgió WebSocket, cuáles son sus principales características y qué ventajas ofrece frente al modelo tradicional basado en HTTP. Sin embargo, comprender una tecnología únicamente desde sus conceptos teóricos no siempre permite dimensionar el impacto que tiene en una aplicación real.

Por este motivo, antes de analizar el funcionamiento interno del protocolo, resulta conveniente observar cómo se comporta WebSocket en un escenario cotidiano. Para ello se utilizará como ejemplo una aplicación de mensajería instantánea, ya que representa uno de los casos de uso más conocidos de la comunicación en tiempo real.

Escenario de estudio

Aplicación de chat donde los usuarios intercambian mensajes de manera simultánea.

Franco Fabricio Joaquín Iara

\ | | /

\ | | /

Servidor WebSocket

Cada usuario accede a la aplicación desde su navegador y espera recibir los mensajes enviados por los demás prácticamente de forma inmediata. El objetivo del sistema consiste en mantener sincronizados a todos los participantes sin necesidad de recargar la página o realizar consultas constantes al servidor.

El usuario ingresa a la aplicación

Cuando Franco abre la aplicación, el navegador aún no utiliza WebSocket. Como ocurre en cualquier sitio web, inicialmente se establece una comunicación mediante HTTP para descargar los recursos necesarios: el documento HTML, las hojas de estilo, los archivos JavaScript y demás elementos que conforman la interfaz.

Franco (Navegador)

HTTP

|



Servidor Web

Una vez descargados estos recursos, la aplicación ya puede ejecutarse en el navegador del usuario.

Se establece el canal de comunicación

Con la aplicación ya cargada, el código JavaScript solicita crear una conexión WebSocket con el servidor. A partir de este momento comienza a existir un canal de comunicación permanente entre ambas partes, el cual permanecerá disponible durante toda la sesión del usuario. Este canal será utilizado para enviar y recibir todos los mensajes posteriores, evitando la necesidad de crear una nueva conexión para cada intercambio de información.

Franco



Servidor

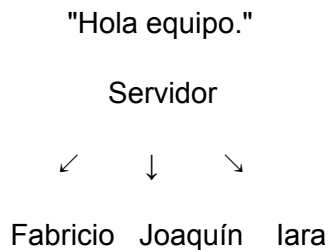
Un usuario envía un mensaje

Supongamos ahora que Franco escribe el siguiente mensaje: *Hola equipo*.

Al presionar el botón **Enviar**, el navegador transmite inmediatamente el mensaje utilizando la conexión WebSocket que ya se encontraba establecida. No es necesario realizar una nueva petición HTTP ni esperar que el servidor sea consultado nuevamente. El mensaje simplemente viaja por el canal de comunicación existente.

El servidor distribuye la información

Cuando el servidor recibe el mensaje, lo procesa e identifica qué usuarios deben recibirlo. Luego lo distribuye inmediatamente hacia todos los clientes conectados. Este mecanismo, conocido como **broadcast**, permite que múltiples usuarios reciban la misma información prácticamente al mismo tiempo. Gracias a este comportamiento, todos los participantes observan la actualización de forma casi instantánea.



¿Qué acabamos de observar?

Aunque el ejemplo es sencillo, permite identificar varias de las características que hacen de WebSocket una tecnología adecuada para aplicaciones de comunicación en tiempo real.

Durante todo el proceso:

- La página nunca necesitó recargarse.
- El servidor pudo enviar información sin esperar nuevas solicitudes HTTP.
- La conexión permaneció abierta durante toda la comunicación.
- Los mensajes llegaron prácticamente de forma inmediata a todos los usuarios.
- El intercambio de información ocurrió utilizando una única conexión entre cliente y servidor.

Funcionamiento interno de WebSocket

En el capítulo anterior se analizó el funcionamiento de una aplicación de mensajería en tiempo real basada en WebSocket. Se observó cómo los usuarios pueden intercambiar mensajes de forma inmediata gracias a una conexión permanente entre el cliente y el servidor. Sin embargo, aún queda una pregunta por responder: **¿cómo consigue WebSocket mantener esa conexión y transmitir información de forma tan eficiente?**

Establecimiento de la conexión (Handshake)

El primer paso para utilizar WebSocket consiste en establecer una conexión entre el cliente y el servidor. Sin embargo, esta comunicación **no comienza directamente utilizando el protocolo WebSocket**, sino que inicialmente se realiza mediante una solicitud HTTP especial.

Este proceso recibe el nombre de **Handshake** (o "apretón de manos") y tiene como objetivo que ambas partes acuerden abandonar el protocolo HTTP y comenzar a comunicarse mediante WebSocket. A diferencia de HTTP, donde cada solicitud es independiente, este intercambio inicial ocurre **una única vez** durante toda la sesión.

El flujo del proceso puede observarse en la siguiente figura:

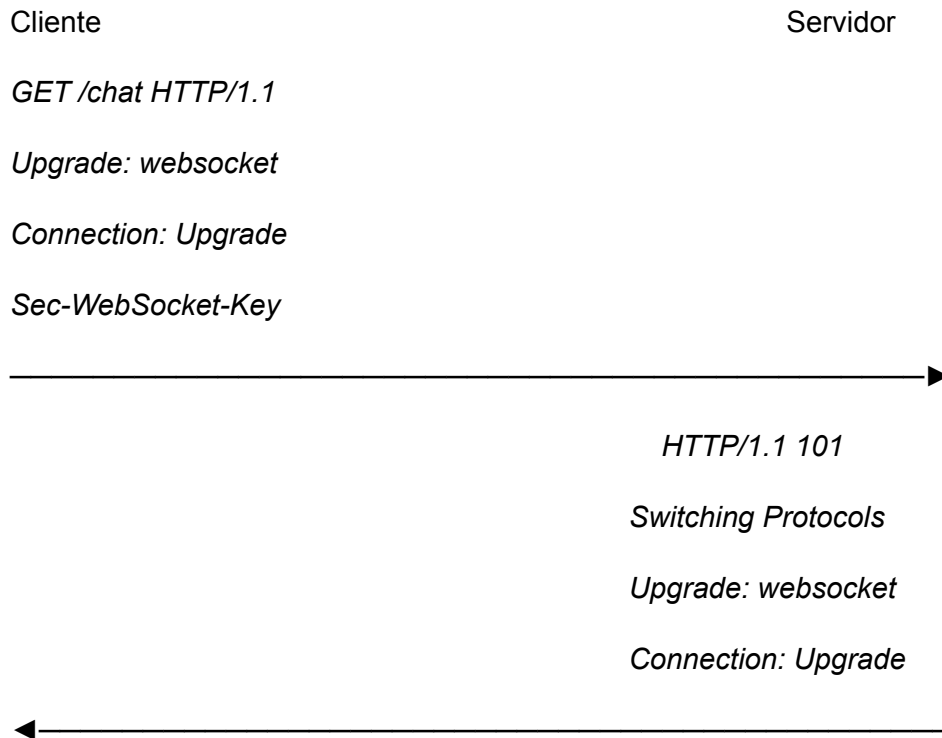


Figura 7.1. Proceso de establecimiento de una conexión WebSocket mediante el Handshake. El cliente solicita el cambio de protocolo utilizando una petición HTTP especial y el servidor responde con el código 101 Switching Protocols, confirmando el inicio de la comunicación WebSocket.

Paso 1. Solicitud del cliente

El proceso comienza cuando el navegador envía una petición HTTP al servidor. A simple vista parece una solicitud HTTP convencional, pero incorpora una serie de encabezados adicionales que indican la intención de cambiar el protocolo de comunicación. Los encabezados más importantes son:

- **Upgrade: websocket**, que solicita explícitamente utilizar el protocolo WebSocket.
- **Connection: Upgrade**, que indica que la conexión actual será reutilizada para cambiar de protocolo, en lugar de cerrarse al finalizar la respuesta HTTP.
- **Sec-WebSocket-Key**, una clave generada aleatoriamente por el cliente que permite al servidor verificar que la solicitud corresponde realmente a una conexión WebSocket y no a una petición HTTP común.

Paso 2. Respuesta del servidor

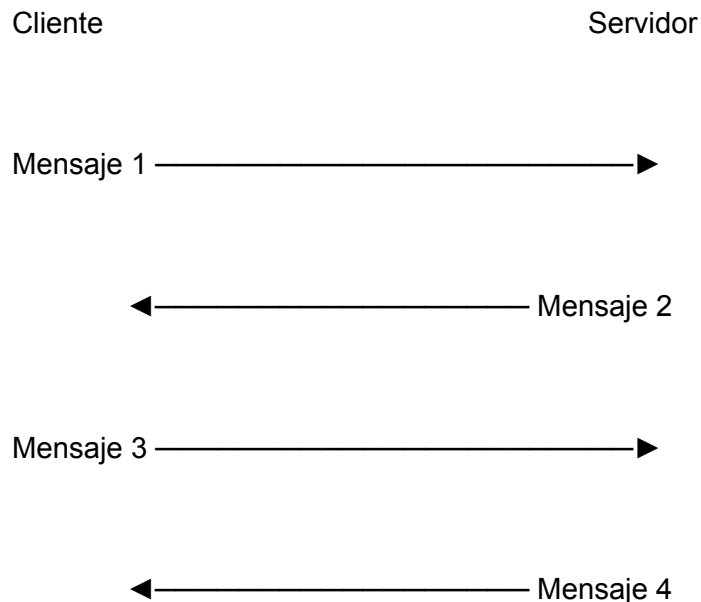
Si el servidor admite conexiones WebSocket y acepta la solicitud recibida, responde con el código de estado **HTTP 101 Switching Protocols**. Este código tiene un significado muy importante: indica que el servidor acepta abandonar el protocolo HTTP y cambiar al protocolo WebSocket. Junto con el código 101, el servidor devuelve nuevamente los encabezados **Upgrade** y **Connection**, confirmando que ambas partes utilizarán el nuevo protocolo a partir de ese momento.

Paso 3. Cambio de protocolo

Una vez finalizado el Handshake, la negociación concluye y ya no vuelven a intercambiarse mensajes HTTP dentro de esa conexión. Desde este instante, cliente y servidor comienzan a enviar y recibir información mediante el protocolo WebSocket utilizando un único canal de comunicación que permanecerá abierto hasta que alguno de los extremos decida finalizar la sesión.

Intercambio de mensajes mediante Frames

Una vez establecida la conexión, ambos participantes pueden enviar mensajes en cualquier momento. Los datos ya no viajan como solicitudes HTTP completas, sino mediante **frames**, estructuras mucho más pequeñas y eficientes diseñadas específicamente para el protocolo WebSocket.



Esta capacidad de enviar información en ambas direcciones de forma simultánea es lo que se conoce como comunicación **Full-Duplex**. Gracias a ello, el servidor puede informar inmediatamente la ocurrencia de un evento sin esperar una nueva solicitud del cliente.

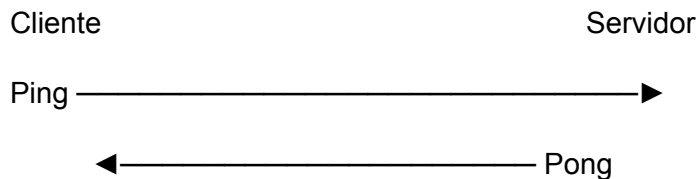
Mantenimiento de la conexión

En conexiones que permanecen abiertas durante largos períodos de tiempo, resulta importante verificar que ambas partes continúan disponibles.

Para ello, WebSocket incorpora los mensajes de control **Ping** y **Pong**.

Periódicamente uno de los extremos envía un mensaje **Ping**.

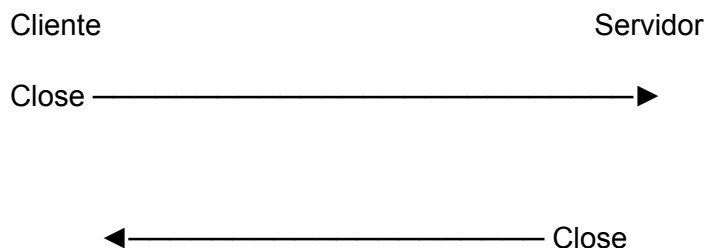
Si la otra parte continúa conectada, responde automáticamente con un mensaje **Pong**.



Este mecanismo permite detectar desconexiones inesperadas y mantener el estado de la comunicación.

Finalización de la conexión

Cuando cliente o servidor desean finalizar la comunicación, envían un **Close Frame**. Luego de intercambiar estos mensajes, ambos liberan los recursos asociados a la conexión.



Conexión finalizada

Aplicaciones, escenarios de uso y consideraciones de WebSocket

Uno de los principales objetivos de este informe consiste en determinar **en qué situaciones una tecnología aporta un beneficio real y cuándo resulta preferible optar por otra alternativa**. En este capítulo se presentan algunos de los escenarios donde WebSocket demuestra sus principales fortalezas.

Aplicaciones de mensajería instantánea: Las aplicaciones de chat representan uno de los casos de uso más conocidos de WebSocket. En este tipo de sistemas, los usuarios esperan que los mensajes enviados sean recibidos por los demás participantes prácticamente de forma inmediata. Gracias a la comunicación bidireccional y persistente que ofrece WebSocket, el servidor puede distribuir un mensaje hacia todos los usuarios conectados en el mismo instante en que lo recibe, proporcionando una experiencia mucho más fluida.

Videojuegos multijugador: Los videojuegos en línea requieren una actualización constante del estado de la partida. La posición de los jugadores, los movimientos realizados, los disparos, los cambios en el escenario y otros eventos deben sincronizarse continuamente entre todos los participantes. En este contexto, WebSocket permite transmitir los cambios casi en tiempo real mediante una única conexión persistente, reduciendo la latencia y mejorando la experiencia de juego.

Herramientas colaborativas: Aplicaciones como editores de documentos, pizarras virtuales o plataformas de trabajo colaborativo. Cuando uno de los participantes realiza un cambio, los demás deben visualizarlo inmediatamente para mantener todos los documentos sincronizados. La comunicación permanente proporcionada por WebSocket permite que el servidor distribuya las modificaciones apenas ocurren, evitando inconsistencias y favoreciendo el trabajo colaborativo.

Paneles de monitoreo y visualización de datos: Muchas organizaciones utilizan paneles de control donde se muestran métricas que cambian constantemente, como consumo de recursos, estado de servidores, información bursátil, sensores industriales o indicadores de producción. En estos casos, el servidor puede enviar las nuevas mediciones automáticamente cada vez que se producen, evitando que el navegador deba consultar repetidamente si existen actualizaciones disponibles. Esto reduce el tráfico de red y mejora la capacidad de respuesta de la aplicación.

Sistemas de notificaciones en tiempo real: Otro escenario frecuente corresponde a las aplicaciones que necesitan informar eventos inmediatamente después de que ocurren. Entre ellos pueden mencionarse: Notificaciones de nuevas publicaciones, Alertas de seguridad, Confirmaciones de operaciones, Actualizaciones de pedidos, Avisos de disponibilidad de recursos. Mediante WebSocket, el servidor puede enviar estas notificaciones en el momento exacto en que se generan, sin depender de consultas periódicas realizadas por el cliente.

Consideraciones para su implementación.

Si bien WebSocket ofrece numerosas ventajas para aplicaciones que requieren comunicación en tiempo real, su utilización también presenta ciertas limitaciones que deben considerarse antes de adoptarlo como solución para un proyecto.

Consumo de recursos: Una de las principales características de WebSocket es mantener una conexión persistente entre el cliente y el servidor. Sin embargo, cada conexión activa consume memoria y otros recursos del servidor. En aplicaciones con miles de usuarios conectados simultáneamente, este aspecto debe contemplarse durante el diseño de la infraestructura para garantizar un funcionamiento adecuado.

No todas las aplicaciones necesitan comunicación permanente: Muchas aplicaciones web realizan únicamente consultas ocasionales al servidor, como sitios institucionales, blogs, tiendas en línea o sistemas de gestión tradicionales. En estos casos, mantener una conexión WebSocket abierta durante toda la sesión no aporta beneficios significativos y puede representar un consumo innecesario de recursos. Para este tipo de aplicaciones, HTTP continúa siendo una alternativa más simple y eficiente.

Mayor complejidad en el despliegue: Aunque implementar WebSocket resulta relativamente sencillo utilizando bibliotecas como Socket.IO, desplegar aplicaciones con un gran número de conexiones concurrentes puede requerir configuraciones adicionales en balanceadores de carga, proxies inversos o servidores distribuidos. En proyectos de gran escala suele ser necesario incorporar mecanismos que permitan compartir el estado de las conexiones entre múltiples servidores.

Dependencia de una conexión estable: Al tratarse de una comunicación persistente, el funcionamiento de WebSocket depende de que la conexión entre cliente y servidor permanezca activa. Si se produce una interrupción de la red, la comunicación se pierde y la aplicación debe detectar la desconexión y establecer nuevamente el canal de comunicación.

No reemplaza completamente a HTTP: WebSocket fue diseñado para resolver un problema específico: la comunicación bidireccional en tiempo real. Sin embargo, la mayoría de las operaciones habituales de una aplicación web, como autenticación, carga de páginas, descarga de archivos o consumo de APIs REST, continúan realizándose mediante HTTP. Por este motivo, ambas tecnologías suelen utilizarse de forma complementaria dentro de una misma aplicación.

Comparación de WebSocket con otras tecnologías de comunicación en tiempo real.

Aunque todas estas tecnologías permiten implementar mecanismos de comunicación en tiempo real, cada una fue diseñada para resolver necesidades diferentes. Por este motivo, la comparación no busca determinar cuál es superior, sino identificar en qué situaciones resulta más conveniente utilizar cada una.

WebSocket se destaca cuando la aplicación requiere una comunicación continua y bidireccional entre el navegador y el servidor. Gracias a su conexión persistente, permite intercambiar mensajes con baja latencia y resulta especialmente adecuado para aplicaciones donde los usuarios generan y reciben información de manera constante (Chats, Juegos, Colaboración, Dashboards).

Server-Sent Events (SSE), por el contrario, ofrece un mecanismo más simple cuando únicamente es necesario que el servidor envíe información al cliente. En aplicaciones donde los usuarios solo reciben actualizaciones —como paneles informativos o sistemas de

noticias—, SSE puede representar una alternativa más sencilla de implementar (Notificaciones y actualizaciones unidireccionales).

WebRTC fue desarrollado para escenarios donde la comunicación debe establecerse directamente entre dispositivos, sin que el servidor intervenga en el intercambio principal de datos. Esta característica lo convierte en la opción más apropiada para videollamadas, conferencias y transmisión de audio o video en tiempo real (Videollamadas, audio, video y compartición de archivos).

Por su parte, MQTT fue diseñado pensando en dispositivos con recursos limitados y comunicaciones de bajo consumo. Su arquitectura basada en un Broker y en el modelo de publicación/suscripción facilita la conexión de miles de sensores y dispositivos, siendo ampliamente utilizado en aplicaciones de Internet de las Cosas (IoT) (IoT, sensores, automatización).

Conclusiones

A lo largo de la investigación se observó que WebSocket surge como una respuesta a las limitaciones del modelo tradicional basado en HTTP cuando una aplicación requiere un intercambio continuo de información entre el cliente y el servidor. Gracias al establecimiento de una conexión persistente y bidireccional, es posible reducir la latencia, disminuir la sobrecarga de las comunicaciones y ofrecer una experiencia de usuario más fluida en aplicaciones donde la información cambia constantemente.

Comprobamos estos conceptos en un escenario práctico, evidenciando que la comunicación se realiza de manera inmediata y sin necesidad de establecer nuevas solicitudes HTTP para cada mensaje intercambiado. Esto confirmó que WebSocket constituye una alternativa eficiente para aplicaciones como sistemas de mensajería instantánea, videojuegos multijugador, herramientas colaborativas y paneles de monitoreo en tiempo real.

Sin embargo, durante la investigación también se comprobó que WebSocket no representa una solución universal. Existen escenarios en los que otras tecnologías ofrecen características más adecuadas para resolver problemas específicos. Server-Sent Events simplifica la implementación cuando únicamente es necesario enviar información desde el servidor hacia el cliente; WebRTC resulta más apropiado para comunicaciones directas entre usuarios, como videollamadas; mientras que MQTT ofrece importantes ventajas en aplicaciones de Internet de las Cosas (IoT), donde participan numerosos dispositivos con recursos limitados.

En consecuencia, la principal conclusión obtenida durante este informe es que la elección de una tecnología de comunicación en tiempo real no debe basarse únicamente en sus características técnicas, sino en el problema que se desea resolver. Desde esta perspectiva, WebSocket demostró ser una solución sólida, eficiente y ampliamente adoptada para aplicaciones web que requieren comunicación bidireccional de baja latencia entre clientes y servidores.

Finalmente, esta investigación permitió no solo adquirir conocimientos sobre el protocolo WebSocket, sino también comprender la importancia de analizar distintas alternativas tecnológicas antes de incorporarlas a un proyecto de software. La experiencia obtenida durante la implementación y comparación realizada reafirma el valor de las Pruebas de Concepto como herramienta para fundamentar decisiones de diseño y seleccionar la tecnología más adecuada según las necesidades de cada sistema.

Bibliografías

<https://www.ibm.com/docs/es/was/9.0.5?topic=applications-websocket>

<https://developer.mozilla.org/es/docs/Web/API/WebSocket>

<https://socket.io/docs/v4/>